TEK4090 – Introduction to Modern Control
Assignment 2 v4.0

Due: October 23 2024, 13:15

---

1. (10 points) Consider the continuous-time system,

$$\ddot{x} = u \tag{1}$$

$$J(t_0) = \frac{1}{2}x^2(0) + r\int_0^2 u^2 dt. \tag{2}$$

Hint: read §6.3 in Lewis, Syrmos, & Vraibie for some relevant example problems

(a) Write the Hamilton-Jacobi-Bellman (HJB equation), eliminating $u(t)$

Ans: We begin by writing the hamiltonion equation:

$$H(x, u, \lambda, t) = ru^2 + \lambda^T f(x, u, t)$$

Where $\lambda \in \mathbb{R}^2$. We define f as a vector in $\mathbb{R}^2$ as well where

$$f = \begin{bmatrix} x_2 \\ u \end{bmatrix}$$

if we take $\dot{x} = x_2$ and $\dot{x_2} = u$. We not get that the Hamiltonion is

$$H = ru^2 + \lambda_1 x_2 + \lambda_2 u$$

Our goal is to find the Hamilton Jacobi equation,

$$-J_t^* = min_u \left(ru^2 + \lambda_1 x_2 + \lambda_2 u\right)$$

which we find by optimizing H for u and substituting the optimal u in H. First note that $\frac{\partial^2 H}{\partial u^2} = 2r \geq 0$, which is a condition for global convexity, assuming $r \geq 0$. This means that the optimal u^* we find is the global minimum of the Hamiltonion.

$$\begin{aligned} \frac{\partial H}{\partial u} &= 0 \\ &= 2ru + \lambda_2 \\ u^* &= -\frac{\lambda_2}{2r} \end{aligned}$$

We now substitute optimal control into h

$$\begin{aligned} H^* &= r\left(-\frac{\lambda_2}{2r}\right)^2 + \lambda_1 x_2 - \frac{\lambda_2^2}{2r} \\ &= \frac{\lambda_2}{4r} + \lambda_1 x_2 - \frac{\lambda_2^2}{2r} \\ &= \frac{-\lambda_2^2}{4r} + \lambda_1 x_2 \end{aligned}$$

Substituting this into J^* we get that

$$-J_t^* = min_u\left(ru^2 + \lambda_1 x_2 + \lambda_2 u\right) = \frac{-\lambda_2^2}{4r} + \lambda_1 x_2$$

(b) Assume that the optimal cost-to-go function $J^*(t)$ is given by,

$$J^*(t) = \frac{1}{2}s_1(t)x^2(t) + s_2(t)x(t)\dot{x}(t) + \frac{1}{2}s_3(t)\dot{x}^2(t). \tag{3}$$

Use the HJB equation to derive coupled differential equations for these $s_i(t)$'s, and find boundary conditions for these equations.

Ans: We find that

- $\lambda_1 = \frac{\partial J^*}{\partial x} = s_1 x + s_2\dot{x}$
- $\lambda_2 = \frac{\partial J^*}{\partial \dot{x}} = s_2 x + s_3\dot{x}$
- $\frac{\partial J^*}{\partial t} = \frac{1}{2}s_1 x^2 + s_2 x\dot{x} + \frac{1}{2}s_2\dot{x}^2$

If we plug this into the HJB equation we found from a), we get

$$\begin{aligned} -\left[\frac{\partial J^*}{\partial t} = \frac{1}{2}\dot{s_1}x^2 + \dot{s_2}s_2 x\dot{x} + \frac{1}{2}\dot{s_3}\dot{x}^2\right] &= \frac{-1}{4r}\left(s_2 x + s_3\dot{x}\right)^2 + \left(s_1 x + s_2\dot{x}\right)\dot{x} \\ &= \frac{-1}{4r}\left(s_2^2 x^2 + 2s_2 s_3 x\dot{x} + s_3^2\dot{x}^2\right) + s_1 x\dot{x} + s_2\dot{x}^2 \\ &= \frac{-s_2^2 x^2}{4r} + x\dot{x}\left(s_1 - \frac{s_2 s_3}{2r}\right) + \dot{x}^2\left(s_2 - \frac{s_3^2}{4r}\right) \end{aligned}$$

Now we can match the coefficients with the original J^* and get

$$-\frac{1}{2}\dot{s_1}x^2 - \dot{s_2}x\dot{x} - \frac{1}{2}\dot{s_3}\dot{x}^2 = \frac{-s_2^2 x^2}{4r} + x\dot{x}\left(s_1 - \frac{s_2 s_3}{2r}\right) + \dot{x}^2\left(s_2 - \frac{s_3^2}{4r}\right)$$

- $-\frac{1}{2}\dot{s_1} = -\frac{s_2^2}{4r}$
- $-\dot{s_2} = s_1 - \frac{s_2 s_3}{2r}$
- $-\frac{1}{2}\dot{s_3} = s_2 - \frac{s_3^2}{4r}$

Which makes

- $\dot{s_1} = \frac{s_2^2}{2r}$
- $\dot{s_2} = \frac{s_2 s_3}{2r} - s_1$
- $\dot{s_3} = \frac{s_3^2}{2r} - 2s_2$

Now for the initial conditions. At $t = 2 = T$, cost to go is $J^*(T) = \frac{1}{2}x(T)^2$. From $J^* = \frac{1}{2}s_1(T)x^2 + s_2(T)x\dot{x} + s_3(T)\dot{x}^2$, we draw that

- $s_1(2) = 1$
- $s_2(2) = 0$
- $s_3(2) = 0$

(c) Write the linear state feedback optimal control policy in terms of the $s_i(t)$'s.

Ans: We have $u^* = \frac{-\lambda_2}{2r}$ from the HJB.
$\lambda_2 = s_2 x + s_3 \dot{x}$. So

$$u^* = \frac{-s_2 x}{2r} - \frac{s_3 \dot{x}}{2r}$$

This is the linear state feedback. Its the optimal control policy in terms of s_2 and s_3, with gains $\frac{-s_2(t)}{2r}$ on position and $\frac{-s_3(t)}{2r}$ on velocity.

2. (10 points) Pick a system from a paper that you have found on a topic that you are interested in, potentially the topic you will choose for your final project.

(a) What is/are the paper(s) you are drawing from?

Ans: This paper presents a dynamic model of a quadcopter UAV with friction effects, and a two level input-output control strategy combining linearization and PID controllers

(b) What are the dynamics of the system in question? What are the inputs and outputs?

Ans:

- ***Inputs:***
  1. Total thrust F
  2. Torques $[\rho_\phi, \rho_\theta, \rho_\psi]$
- ***Outputs:***
  - Position $\xi = [x, y, z]^T$
  - Orientation $\eta = [\phi, \theta, \psi]^T$

The quadcopter is modelled as a nonlinear underactuated system. The euler-lagrange equations are used to derive the equations of motion, and they also include frictionless terms.

By friction terms, they mean simplified models for including air resistance and aerodynamic drag

(c) What is the system trying to achieve?

Ans: The paper tries to solve the following

1. Hovering, take-off and landing
2. Follow a predetermined path in 3d, or geometric trajectories
3. Stabilizing attitude, namely the pitch, roll and yaw.

(d) What are the challenges for the system?

Ans: The problems in the paper are

- ***Problem 1***: Coupling between rotational and translational motions, meaning that in order to move forward, the nose must first tip down, as the only thrusters points down.
- ***Problem 2***: Nonlinear dynamics. This means that there is not a function that can connect the inputs to the outputs, like a traditional function $f(x) = kx^n$
- ***Problem 3***: Avoiding local linearization, i.e. when changing an action, from hovering to travelling, the local linearization becomes invalid. This is the same logic as using Taylor expansions of a function

around a point. The approximation works great locally, but move away and the error between approximation and true f grows large.

- ***Problem 4***: Friction and air drag. Also in a realistic situtation, you need to figure out how to account for essentially random gusts of wind that puts the drone out of balance.
- ***Problem 5***: Need for fast and stable inner-loop control to support outer loop control
- ***Problem 6***: Singularities when $\phi, \theta = \pm\frac{\pi}{2}$

(e) What control tools can help the system overcome the challenges in achieving its goals?

Ans: For choosing the right gains, they propose that the gains follow a characteristic equation. They do this by pole placement, and finding the gains K that solves the characteristic equation.

For the singularities for the angles ϕ and θ, they define angle limiters, which is simple enough. They could also use quaternions, for more continuous movement, but avoided it for simplicity.

To solve the stability problem, they make an inner- and outer loop. The inner loop controls the quadcopters orientation and attitute, while the outer control deals with the position.

For the input-output linearisation, the paper proposes to transform the non-linear dynamics into linear decoupled systems. Unlike taylor series, which only works locally, this method is globally valid, as each degree of freedom to be tuned is not a function of another variable.

3. (20 points) Suppose that $f : \mathbb{R} \mapsto \mathbb{R}$ is convex, and $a < b$ are in the domain of f.

(a) Show that

$$f(x) \leq \frac{b-x}{b-a} f(a) + \frac{x-a}{b-a} f(b) \tag{4}$$

for all $x \in [a, b]$

Ans: A criteria for a function to be convex is that

$$f(x) \leq \lambda f(a) + (1-\lambda) f(b)$$

$\forall x \in \mathbb{R}$

If we set $x = \lambda a + (1-\lambda)\, b$ where $\lambda = \frac{b-x}{b-a}$, then

$$\begin{aligned} 1 - \lambda &= 1 - \frac{b-x}{b-a} \\ &= \frac{(b-a)-(b-x)}{b-a} \\ &= \frac{x-a}{b-a} \end{aligned}$$

And we get that

$$f(x) = f(\lambda a + (1-\lambda)\, b) \leq \lambda f(a) + (1-\lambda)\, f(b)$$

Which is the criteria for a function to be convex, and it is proved.

(b) Show that

$$\frac{f(x)-f(a)}{x-a} \leq \frac{f(b)-f(a)}{b-a} \leq \frac{f(b)-f(x)}{b-x} \tag{5}$$

for all $x \in (a, b)$. Draw a sketch that illustrates this inequality

Ans: If we pick up our old calculus text book, we find that a function $f : \mathbb{R} \to \mathbb{R}$ iff. $a \leq b \forall c \in \mathbb{R}$

$$\frac{f(c)-f(a)}{c-a} \leq \frac{f(b)-f(c)}{b-c}$$

If we assume that f is convex, then the line between the points $(a, f(a))$ and $(b, f(b))$ will never be under the point $(c, f(c))$.

1. Lets now denote the slope of the line $d\left((a, f(a)), (b, f(b))\right)$ as k. In this case, we use the $\frac{rise}{run}$ formula to get that $\frac{f(b)-f(a)}{b-a} = k$
2. The line $d\left((c, f(c)), (a, f(a))\right)$ as the line $\frac{f(c)-f(a)}{c-a} = k_1$
3. And finally, the line $d\left((b, f(b)), (c, f(c))\right)$ as the line $\frac{f(b)-f(c)}{b-c} = k_2$

If f is truly convex, then $k_1 \leq k \leq k_2$. Therefore,

$$\frac{f(c)-f(a)}{c-a} \leq \frac{f(b)-f(a)}{b-a} \leq \frac{f(b)-f(c)}{b-c}$$

Now we can just replace c with x, becuase this inequality is valid $\forall x \in (a, b)$

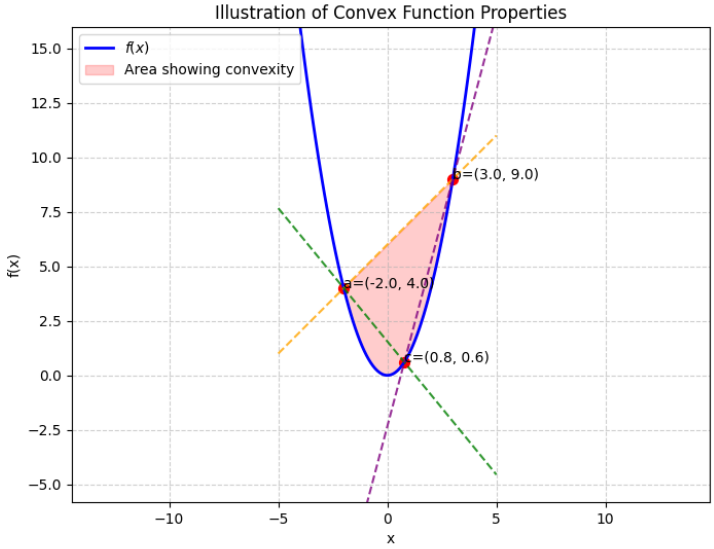


Fig. 1: Building model schematic

(c) Suppose that f is differentiable. Use the result in (b) to show that

$$f'(a) \leq \frac{f(b) - f(a)}{b - a} \leq f'(b). \tag{6}$$

Ans: From b we have that

$$\frac{f(c) - f(a)}{c - a} \leq \frac{f(b) - f(a)}{b - a} \leq \frac{f(b) - f(c)}{b - c}$$

We know from a definition of derivatives in our calculus book, that

$$f'(a) = \lim_{x \to a} \frac{f(x) - f(a)}{x - a}$$

. Then since $x - b \leq 0$ we have that

$$f'(b) = \lim_{x \to b} \frac{f(b) - f(x)}{b - x}$$

From this we get what we wanted to prove, if we allow x to approach both a and b respectively

(d) Suppose f is twice differentiable. Use the result in (c) to show that $f''(a) \geq 0$ and $f''(b) \geq 0$.

Ans: From c), we got that $f'(a) \leq \frac{f(b)-f(a)}{b-a} \leq f'(b)$. Now the second derivative becomes

$$f''(a) = \lim_{h \to 0} \frac{f'(a+h) - f'(a)}{h}$$

Now applying the mean value theorem to the interval $[a, a+h]$, we get $\exists$ som $c \in (a, a+h)$ s.t.

$$f'(c) = \frac{f(a+h) - f(a)}{h}$$

But from our inequality with $x = a + h$, we get that $f'(a) \leq f'(c)$. Since $c > a$ and $f'(c) \geq f'(a)$, the rate of change is non decreasing at a. Therefore,

$$f'(a+h) \geq f'(a)$$

which implies that $\frac{f'(a+h)-f'(a)}{h} \geq 0$. Now taking the limit as h approaches 0 from above, we get

$$\lim_{h \to 0^+} \frac{f'(a+h) - f'(a)}{h} \geq 0$$

which is what we wanted to show. A similar argument is made for $f''(b)$. For $h < 0$, then

$$f(b) \geq f(b+h)$$

so $\frac{f'(b)-f(b+h)}{-h} \geq 0$ for all $h < 0$. Taking the limit as h approaches 0 from below, we get

$$f''(b) = \lim_{h \to 0^-} \frac{f'(b) - f'(b+h}{-h} \geq 0$$

. Therefore, we get that $f''(a) \geq 0$ and $f''(b) \geq 0$

4. (20 points) (LQR for Buildings) Consider a multi-zone building as in Fig. 2.

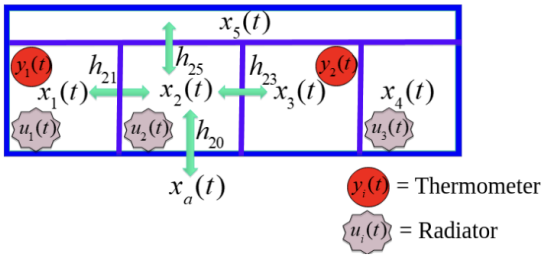


Fig. 2: Building model schematic

(a) Write down the dynamical equations of the building in state-space form, including a term for disturbances $d(t)$, and for the ambient temperature $x_a(t)$, assuming the parameters are

$$\frac{h_{ij}}{C_i} = \begin{cases} 10 \text{ W/°K} & i \text{ is adjacent to } j \\ 0 & \text{else} \end{cases} \tag{7}$$

$$C_i = 1.08 \times 10^6 \text{ J/°K} \tag{8}$$

(b) Consider an LQR cost function of the form

$$\int_0^T \left[(x - \bar{x})^T Q (x - \bar{x}) + u^T R u\right] dt, \tag{9}$$

where $T = 24\text{h} \times 60\frac{\text{m}}{\text{h}} \times 60\frac{\text{s}}{\text{m}} = 86400s$ is one day in seconds. Set $\bar{x} = 20°$, and write the dynamics in the coordinates $z := x - \bar{x}$.

(c) Note that driving the variable $z(t) \to 0$ is therefore equivalent to driving $x(t) \to 20°$. Write a `matlab` script to solve the finite-time horizon LQR problem. Select a positive semi-definite Q and a positive-definite R that gives you "decent" results. Assume the initial conditions, disturbance, and ambient temperature follow:

$$x_i(0) = 15 - i \times \text{sign}\{(-1)^i\} \quad [^\circ \text{ C}] \tag{10}$$

$$d(t) = \max\left[0, 500 \sin\left(\frac{2\pi t}{86400s}\right) - 300\right] \quad [\text{W}] \tag{11}$$

$$x_a(t) = 20° + 5 \sin\left(\frac{2\pi t}{86400s}\right) \quad [^\circ \text{ C}] \tag{12}$$

where the sign function is:

$$\text{sign}(x) = \begin{cases} 1 & x > 0 \\ -1 & x < 0 \\ 0 & x = 0. \end{cases} \tag{13}$$

Note: If solving the finite-time problem proves too difficult, feel free to solve the infinite-time problem instead. `Matlab` has a built-in function called `lqr` that produces the (constant) gain K; read the documentation for details.

(d) Provide:

- A description of your simulation code
- Plots of $d(t)$, $x_a(t)$, $x_i(t)$, and $u(t)$.
- A discussion of the effects of varying Q and R

Ans:

(a) To find the dynamical equations of the building in state-space form, we have to find the system on the form

$$\dot{x} = Ax + Bu, y = Cx$$

The individual room equations are

$$C_i \dot{x_i}(t) = \sum_{j \in \text{neighbors}} h_{ij} \left(x_j(t) - x_i(t)\right) + h_{i0} \left(x_a(t) - x_i(t)\right) + u_i(t) + d_i(t)$$

We rearrange this to get

$$\dot{x} = \sum_j \frac{h_{ij}}{C_i} \left(x_j(t) - x_i(t)\right) + \frac{h_{i0}}{C_i} \left(x_a(t) - x_i(t)\right) + \frac{u_i(t)}{C_i} + \frac{d_i(t)}{C_i}$$

We have 5 rooms, thus 5 states, so $x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ x_4(t) \\ x_5(t) \end{bmatrix}$, 3 controllers that regulate temperature, $u(t) = \begin{bmatrix} u_1(t) \\ u_2(t) \\ u_3(t) \end{bmatrix}$ and finally, one disturbance term for every room $d(t) = \begin{bmatrix} d_1(t) \\ d_2(t) \\ d_3(t) \\ d_4(t) \\ d_5(t) \end{bmatrix}$

We incorporate the disturbance term, ambient temperatures and the controls (radiators) into a single vector $u^* = \begin{bmatrix} u \\ d \\ x_a \end{bmatrix}$

$\frac{h_{ij}}{C_i} = \begin{cases} 10W/^\circ K, \text{ if i adjacent to j} \\ 0, \text{ else} \end{cases}$

This makes. A to be a banded matrix. Since all rooms are adjacent to room 5, the matrix A becomes

$$A = \begin{bmatrix} -\sum_j \frac{h_{1j}}{C_1} & \frac{h_{12}}{C_1} & 0 & 0 & \frac{h_{15}}{C_1} \\ \frac{h_{21}}{C_2} & -\sum_j \frac{h_{2j}}{C_2} & \frac{h_{23}}{C_2} & 0 & \frac{h_{25}}{C_2} \\ 0 & \frac{h_{32}}{C_3} & -\sum_j \frac{h_{3j}}{C_3} & \frac{h_{34}}{C_3} & \frac{h_{35}}{C_3} \\ 0 & 0 & \frac{h_{43}}{C_4} & -\sum_j \frac{h_{4j}}{C_4} & \frac{h_{45}}{C_4} \\ \frac{h_{51}}{C_5} & \frac{h_{52}}{C_5} & \frac{h_{53}}{C_5} & \frac{h_{54}}{C_5} & -\sum_j \frac{h_{5j}}{C_5} \end{bmatrix}$$

$$= \frac{1}{C} \begin{bmatrix} -30 & 10 & 0 & 0 & 10 \\ 10 & -40 & 10 & 0 & 10 \\ 0 & 10 & -40 & 10 & 10 \\ 0 & 0 & 10 & -30 & 10 \\ 10 & 10 & 10 & 10 & -50 \end{bmatrix} \quad C = 1.08 \times 10^6 \text{ J/°K}$$

Since there are three actuators in rooms 1, 2 and 4, the B matrix is written as

$$B = \begin{bmatrix} \frac{1}{C_1} & 0 & 0 \\ 0 & \frac{1}{C_2} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \frac{1}{C_4} \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{1.08\times10^6} & 0 & 0 \\ 0 & \frac{1}{1.08\times10^6} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \frac{1}{1.08\times10^6} \\ 0 & 0 & 0 \end{bmatrix}$$

We have the vector b that takes into account the ambient coupling of temperatures.

$$b = \begin{bmatrix} H_{10} \\ H_{20} \\ H_{30} \\ H_{40} \\ H_{50} \end{bmatrix} = \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{bmatrix}$$

and D is a diagonal matrix with $\frac{1}{C_i} = \frac{1}{1.08\times10^6}$ for disturbance inputs.

If we now follow the notes on building control from the lecture slides of optimal control (lecture 4), we obtain the dynamic equations $\dot{x} = Ax(t) + Bu(t) + bx_a(t) + Dd(t)$

(b) For this problem, we note that $z = x - \bar{x}$, then $x = z + \bar{x}$ and $\dot{x} = \dot{z}$ since $\bar{x}$ is a constant temperature of $20°C \approx 293°K$ since $1°C = 1K$ (K = Kelvin)

so we get the dynamics in the z coordinates as

$$\begin{aligned} \dot{z} &= A(z + \bar{x}) + Bu \\ &= Az + A\bar{x} + Bu \end{aligned}$$

$A\bar{x}$ can be seen as the forcing term towards the reference temperature in the system.

(c) Solving the finite-time horizon problem seems to me very hard, as we dont have any initial points to work backwards from. So I just used the infinite-time horizon

I will attach the matlab script below. From experimenting, I see that a larger value of Q on the diagonals, makes the system more stable, and temperatures in all rooms converge to 20 degrees. Moderate R values allow for moderate control, without spending too much energy. For large values on the diagonal of Q, we prioritize error tracking accuracy. This leads to smaller steady state errors. Values Can go as far as 1000

For the R matrix, we should have smaller values, as far down as 100. What this leads to, is sufficient control action. It means that we dont overdo control, and dont underdo, such that we dont oscillate the control effort.

We see that the Q and R values become weights that account for how important we want to optimize different parts of our system

(d) The plant system from the exercise, $\dot{x} = Ax + Bu$, $y = Cx$. From b), we got that $\dot{z} = Az + A\bar{x} + Bu$. Because of the disturbance terms, we have several terms to include

$$B_d = \begin{bmatrix} 1.08 \times 10^{-6} & 0 & 0 & 0 & 0 \\ 0 & 1.08 \times 10^{-6} & 0 & 0 & 0 \\ 0 & 0 & 1.08 \times 10^{-6} & 0 & 0 \\ 0 & 0 & 0 & 1.08 \times 10^{-6} & 0 \\ 0 & 0 & 0 & 0 & 1.08 \times 10^{-6} \end{bmatrix}$$

$$\text{and } B_u = \begin{bmatrix} 1.08 \times 10^{-6} & 0 & 0 \\ 0 & 1.08 \times 10^{-6} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1.08 \times 10^{-6} \\ 0 & 0 & 0 \end{bmatrix} \text{ and finally } B_a = \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{bmatrix}$$

We are then allowed to rewrite our system as

$$\dot{z} = Az + A\bar{x} + B_d d + B_a x_a$$

$$\dot{z} = Az + B_u u^* + B_d d + \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{bmatrix} \left(293 + 5 \sin\left(\tfrac{2\pi t}{86400s}\right)\right) - 293 * \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{bmatrix}$$

$$\dot{z} = Az + B_u u^* + B_d d + \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{bmatrix} * 5sin\left(\frac{2\pi t}{86400s}\right)$$

The formula for the initial conditional temperature in all 5 rooms is provided, and in degrees they are $(17, 13, 18, 11, 20)$ and in z coordinates, they are $(-4, -7, -2, -9, 0)$

For the LQR system design, I use the lqr function with $Q = I * q$ and $R = I * r$. The difference $(q - r)$ indicates if I favor tracking errors or control effort of the radiators more. In my case, with $q = 1e4$ and $r = 1e2$, the balance is $\frac{q}{r} = 100$.

For observer design, we must check that the system is observable. The LQR controller $u = -Kz$ requires full state knowledge. With only two temperature measurements in two rooms, we need full state observability over all the states $(z)_i^{1 \text{ to } 5}$. We have the true system $\dot{z} = Az + Bu +$ disturbance term, and in parallell, we have a observer running $\dot{z} = Az + Bu + L(y - C\hat{z}$

To derive the augmented matrix, we have the true state dynamic $\dot{z} = Ax + Bu + B_{ext}w$ where $w = \begin{bmatrix} d \\ x_a \end{bmatrix}$ .The observer dynamics is defined as

$$\begin{aligned} \frac{d\hat{z}}{dt} &= A\hat{z} + Bu + L\left(Cz - C\hat{z}\right) \\ &= \hat{z}\left(A - LC\right) + Bu + LCz \end{aligned}$$

The control using the estimates is

$$u = -K\hat{z}$$

Substituting this to the true dynamics gives

$$\dot{z} = Az + B\left(-K\hat{z}\right) + B_{ext}w$$

Also substituting into the observer,

$$\frac{d\hat{z}}{dt} = LCz + \left(A - LC\right)\hat{z} - BK\hat{z}$$

And now pulling it into matrix form we get

$$A_{aug} = \begin{bmatrix} A & -BK \\ LC & A - BK - LC \end{bmatrix}$$

which gives the full augmented system

$$\frac{d}{dt}\begin{bmatrix} z_{true} \\ z_{est} \end{bmatrix} = \begin{bmatrix} A & -BK \\ LC & A - BK - LC \end{bmatrix}\begin{bmatrix} z_{true} \\ z_{est} \end{bmatrix} + \begin{bmatrix} B_{ext} \\ 0 \end{bmatrix} w$$

Now

- A_{aug} is a 10×10 matrix
- B_{aug} is a 10×6 matrix
- z_{aug} is a 10×1 matrix

The initial conditions for z_{est} is $[-2, -2, -2, -2, -2]$. And for z_{true} the initial conditions are $[-4, -7, -2, -9, 0]$ The measurements include sensor noise:

$$y = Cz_{\text{true}} + v, \quad v \sim \mathcal{N}(0, \sigma^2_{\text{meas}} I), \quad \sigma_{\text{meas}} = 0.1^\circ\text{C}$$

Now the plot looks like this, and it should include everything demanded

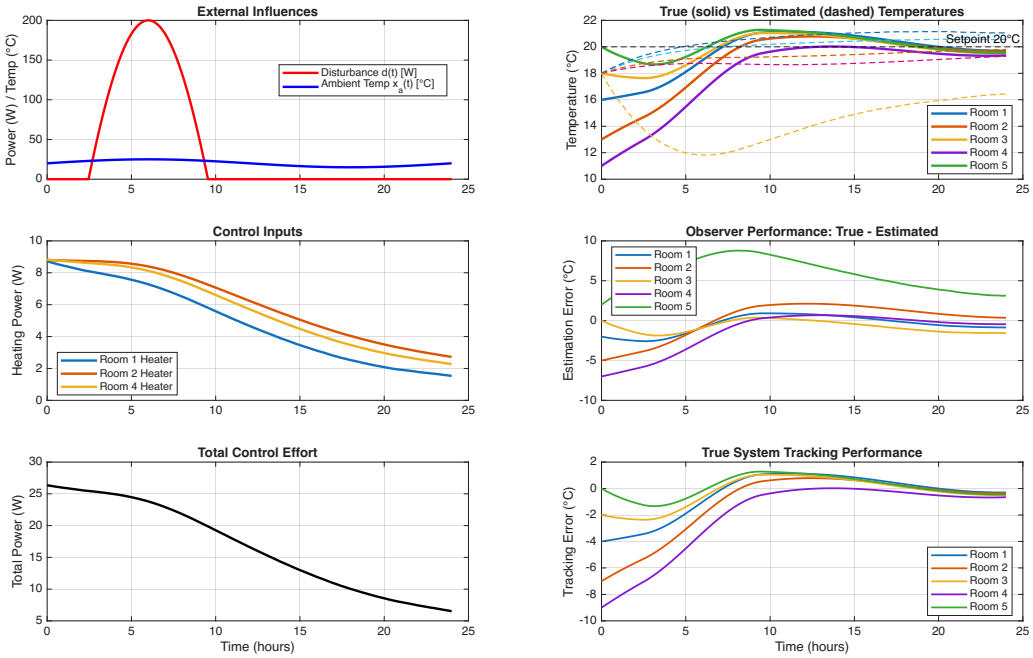


Fig. 3: Building model schematic

This figure makes sense, becase as the initial temperature is lower than the required degree of 20°C, we need a positive control effort from the radiators,

in the form of positive watts, to drive the temperature x to $\bar{x}$. If the initial temperatures were over 20°C, we would get a negative control effort, which just means the radiators are turned off.

5. (10 points) Suppose that $Q = \mathbf{0}$, the zero matrix, and $R = I$. Prove that

$$x_0^T S_0 x_0 = \min_{\{u_k\}_{k=0}^{T-1}} \sum_{k=0}^{T-1} \|u_k\|_2^2, \tag{14}$$

meaning that $J_0 = x_0^T S_0 x_0$ is the minimum control energy required to drive the system to zero.

Ans: We have the discrete LQR with dynamic programming, and an associated cost index

$$J_i = x_N^T S_n x_N + \sum_{k=i}^{N-1} \left(x_k^T Q x_k + u_k^T R u_k \right)$$

we are given that $Q = 0$ and $R = I$. Then we get the cost index

$$J_i = x_N^T S_n x_N + \sum_{k=i}^{N-1} u_k^T u_k$$

We want to find the control sequence u^* that minimizes the cost function J_i. So our problem is the optimization problem

$$J_0^* = min_u J_o(u)$$

s.t. $x_{k+1} = Ax_k + Bu_k$. This is largely already proved on page 270-271 in Optimal Control by Lewis, but I will show the main results. We will have big help of the principle of optimality, which states that along the optimal path, any subpath along that trajectory, is also an optimal trajectory.

So we have that $J_N^* = x_N^T S_N x_N$. Now decrement to N-1 and get $J_{N-1} = u_{N-1}^T u_{N-1} + x_N^T S_N x_N$ and substitute in $x_{k+1} = Ax_k + Bu_k$.

$$J_{N-1} = u_{N-1}^T u_{N-1} + (Ax_{N-1} + Bu_{N-1})^T S_N (Ax_{N-1} + Bu_{N-1})$$

Now we optimize, and since the cost function is convex, we know we find the minimum when $\frac{\partial J_{N-1}}{\partial u} = 0 = u_{N-1} + B^T S_N (Ax_{N-1} + Bu_{N-1})$ and get

$$
\begin{aligned}
0 &= u_{N-1} + B^T S_N A x_{N-1} + B^T S_N B u_{N-1} \\
&= u_{N-1}\left(I + B^T S_N B\right) + B^T S_N A x_{N-1} \\
u^*_{N-1} &= -\left(I + B^T S_N B\right)^{-1} + B^T S_N A x_{N-1}
\end{aligned}
$$

If we now define the gain as

$$K = -\left(I + B^T S_N B\right)^{-1} + B^T S_N A$$

we get that

$$u^*_{N-1} := K_{N-1} x_{N-1}$$

which is the optimal cost to go from step number $N-1$. We can go recursively back with the riccati equations and find that

$$
\begin{aligned}
J^*_0 &= x_0^T S_0 x_0 \\
&= x_N^T S_N x_N + \sum_{k=0}^{N-1} (u^*_k)^T u^*_k \\
&= \sum_{k=0}^{N-1} \|u^*_k\|_2^2
\end{aligned}
$$

Now we just also use the fact that $J^*_0 = min_u J_0(u)$ and get that

$$x_0^T S_0 x_0 = min_u \sum_{k=0}^{N-1} \|u_k\|_2^2$$

6. (10 points) *Crassidis & Junkins* Problem 3.14

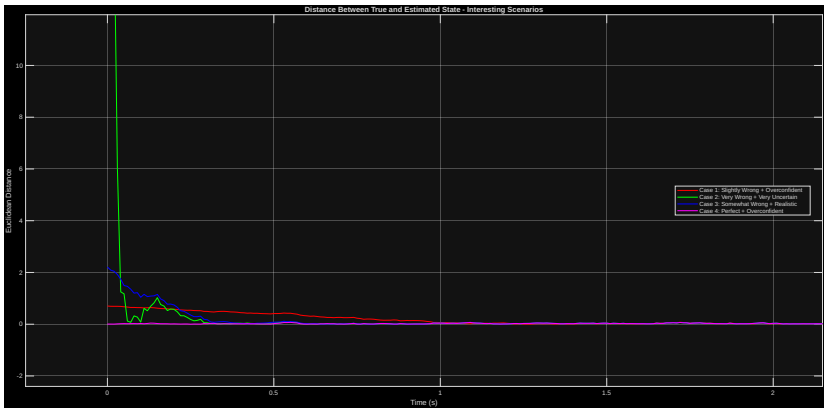
Ans: In this problem, we can imagine we have sensors, and an idea of where we are. The problem is to analyze the difference between what state we think we are in, and the true state. We use two different parameters to analyze the different cases. The initial conditions and the covariance matrix. The initial condition means where we think we are, and the covariance matrix means how sure we are of that. A covariance matrix with 0 as elements, means we are absolutely sure we are in the proper initial condition. So if the distance between the true state

and the measured state is

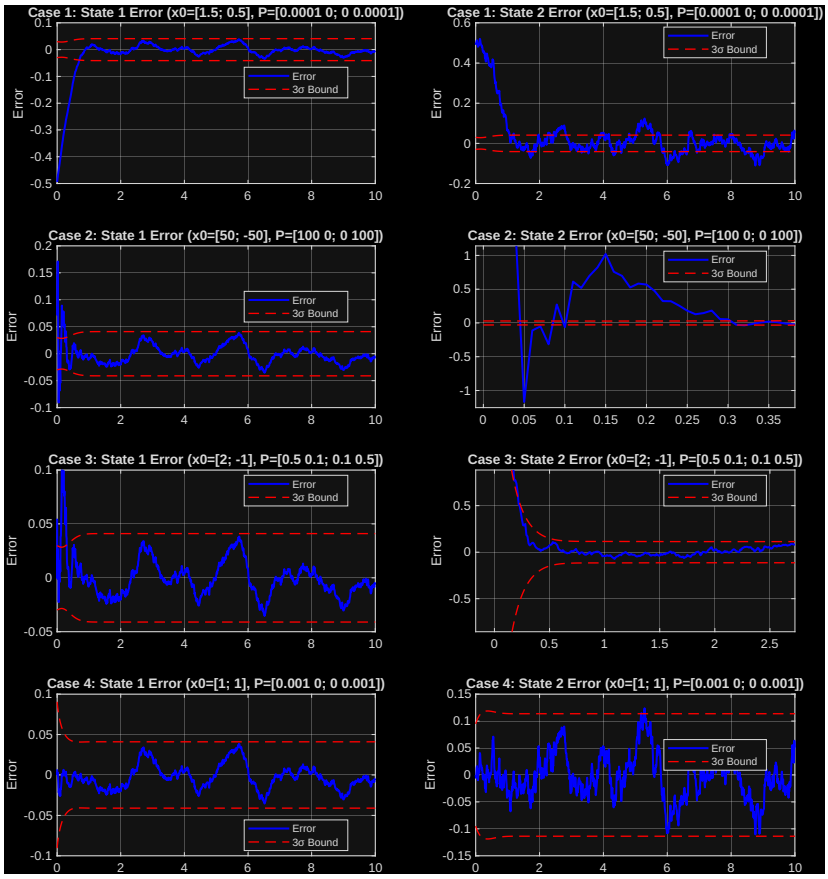
$$\|x_{true} - x_{hat}\| = c$$

Then the speed that c converges to 0, will be determined by the covariance matrix itself.

Here is the plot that shows the distance in norms for several initial conditions and covariance matrices.



And here is the plot that shows the 3sigma bounds



The code is attached in the appendix.

7. (10 points) *Crassidis & Junkins* Problem 3.15

Ans: What we can see from the new H matrix, is that convergence for the (l^2) norms between the states true and predicted states, happens almost instantly, in a matter of miliseconds go to 0. The reason is because of observability. From the last exercise, a covariance matrix of zero is synologous with an overconfidence in the true difference between predicted and actual state, being small.

1. ***Case 1*** Here we are very confident, and also relatively. close to the true state.

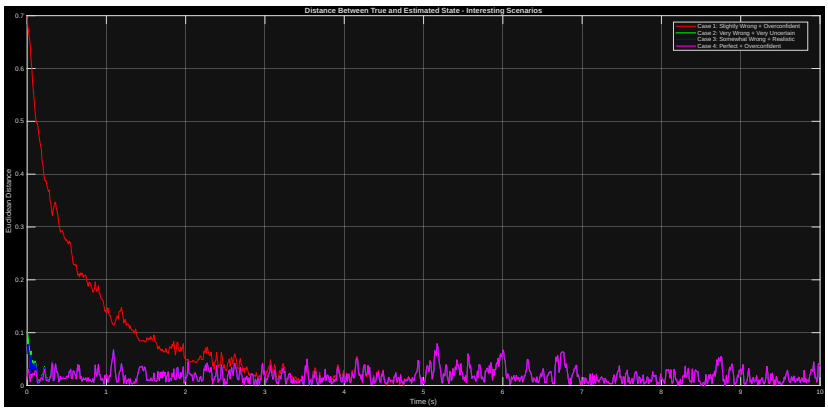
   - For state 1, we see that its taking more time to converge to between the 3σ bounds

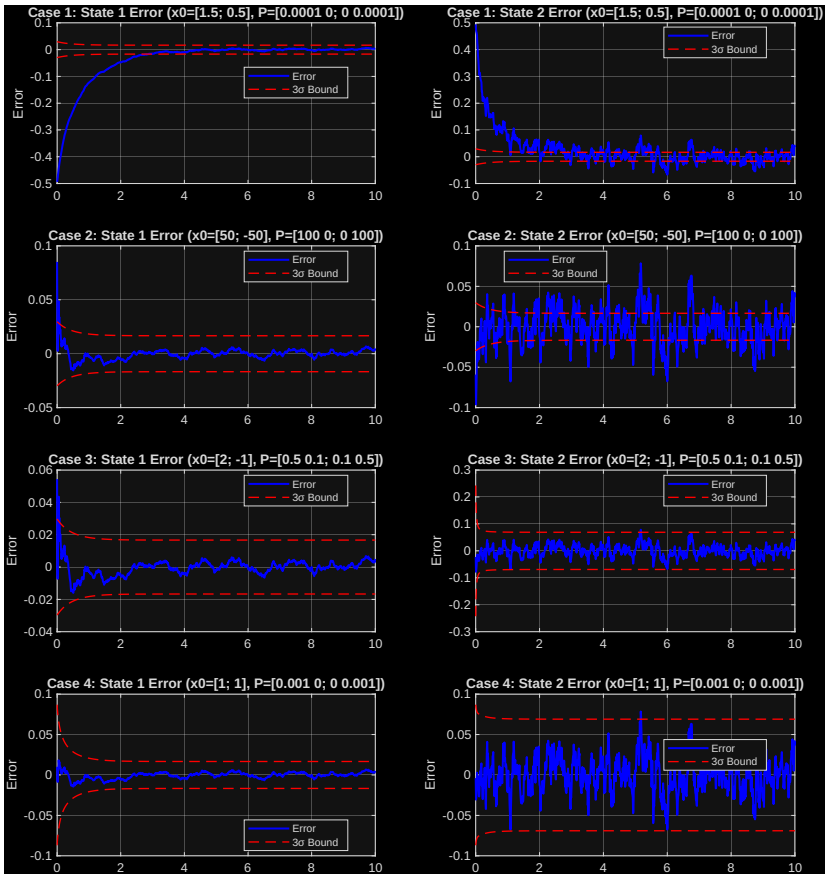
- For state 2, we have the same story, longer time, with higher variance, but it seems to be more within the 3σ bounds

2. ***Case 2*** Here we have low confidence, and far off initial states.
   - For state 1, we have better performance, with tighter 3σ bounds, and also faster convergence.
   - For state 2, its performing alot better, with the previous measurement model having really bad convergence. But there is still a lot of outliers in the data, because its not largely contained within the 3σ bounds
3. ***Case 3*** Here we start off close to the true state, but trust our model more than our sensors.
   - For state 1, we have better performance for convergence and variance. Great success
   - For state 2, same story, faster convergence, and smaller 3σ bounds.
4. ***Case 4*** Here we are spot on, but overconfident.
   - For state 1, we have better convergence and tighter bounds
   - For state 2, we have also better convergence, and smaller variance with slightly tighet bounds

So overall, we can say confidently that the model has largely improved, by providing a measurement model that gives better observability. We would much rather have a realistic balance between overconfidence and low self-esteem in our sensors.

Here are the figures from the matlab script





8. (10 points) *Crassidis & Junkins* Problem 7.4

Ans:

Quick note, Im just ramsing up the important math at the beginning. You can skip to where I say 'Furhtermore' for direct results from simulation. Instead of using euler angles, a nice representation of attitude is using quaternions.

$$\vec{q} = [q_1, q_2, q_3, q_4] = [\epsilon, q_4]$$

where ϵ is the vector part. The kinematics of the quaternion follows

$$\frac{dq}{dt} = \frac{1}{2}\Omega(\vec{\omega})q$$

where ω is the angular velocity. From mechanics, the velocity of a particle rotating about an axis, is derived by $\vec{v} = \vec{\omega} \times \vec{r}$. Not important here. Just meant to illustrate that its a measure of speed, given by radians per unit of time.

For the gyro measurements, we used a first order markov process

$$\vec{\omega}_{measured} = \omega_{true} + \beta + \eta_v \qquad \text{(the measurement noise)}$$

$$\frac{d\beta}{dt} = \eta_u \qquad \text{(bias drift noise)}$$

where beta is the gyro bias vector.

For the error state formulation, we use the direct error to maintain the quaternion normalization constraint, $\delta\vec{q} = \vec{q} \otimes \hat{\vec{q}}^{-1}$

The error state dynamics are given as linearized error dynamics,

- $\frac{d\delta\alpha}{dt} = -[\omega\times]\,\delta\alpha - \Delta\beta - \eta_v$
- $\frac{d\Delta\beta}{dt} = \eta_u$

where $[\omega\times]$ is the skew symmetric matrix given by $\begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}$

And now for the extended kalman filter implementation, we start by defining the state vector $\vec{x} = \left[\vec{q}, \vec{\beta}\right]$. For each step in our discretized timeline, we propagate the new quaternion as

$$\hat{q}^-_{k+1} = \frac{1}{2}\Omega(\hat{\omega}_k \Delta t)\hat{q}^+_k$$

where the exponential is computed as

$$exp\left(\frac{1}{2}\Omega(\omega)\delta t\right) = \begin{bmatrix} cos\,(1/2\|\omega\|\Delta t)\,I - [\phi\times] & \psi \\ -\psi^T & cos\,(1/2\|\omega\|\Delta t) \end{bmatrix}$$

with $\psi = sin\,(1/2\|\omega\|\Delta t) * \frac{\omega}{\|\omega\|}$

For simplification, I used a diagonal process noise covariance matrix.

$$Q = \begin{bmatrix} \sigma_v^2 \Delta t \cdot I_3 & 0 \\ 0 & \sigma_u^2 \Delta t \cdot I_3 \end{bmatrix}$$

And is in this case a 6×6 shaped matrix

The sensor measurement updates are given by

$$b_i = A(q)r_i + v_i$$

for i = 1,2,3. $A(q)$ is the attitude matrix and r_i are the reference vectors. The measurement matrix is given by

$$H = \begin{bmatrix} [A(\hat{q})r_1\times] & 0 \\ [A(\hat{q})r_2\times] & 0 \\ [A(\hat{q})r_2\times] & 0 \end{bmatrix}$$

where again $[v\times]$ is the skew symmetric matrix.

Furthermore, for the implementation strategy, we assume a constant angular velocity, and measurements are made with gyros that contain a bias. The angular velocity is given by $\omega_{true} = [0.001, 0.002, 0.0005]$ rad /s. This creates a slow and accumulating rotation of all the axises, which is an attempt of more realistic dynamics. This is implemented with the function exact attitude. I also assume constant gyro biases $\beta_{true} = [0.1, 0.1, 0.1]$ deg/hr $= [4.848 * 10^{-7}, 4.848 * 10^{-7}, 4.848 * 10^{-7}]$ rad/s

The sampling rate for the gyroscope is 1 Hz, which makes $dt = 1$ second. I put $T = 60$ minutes, and then I put the discretization as $N = \frac{T}{dt}$. Thus for smaller dt, I increase the number N

Now we can start analyzing the filter performance

0.1 baseline scenario

This scenario represents balanced and realistic sensor performance.

- $\sigma_u = 1 * 10^{-5}$ rad/s/$\sqrt{\text{Hz}}$
- $\sigma_v = 1 * 10^{-4}$ rad/s/$\sqrt{\text{Hz}}$

0.2 high σ_v

This scenario tests sensitivity to gyro measurement noise

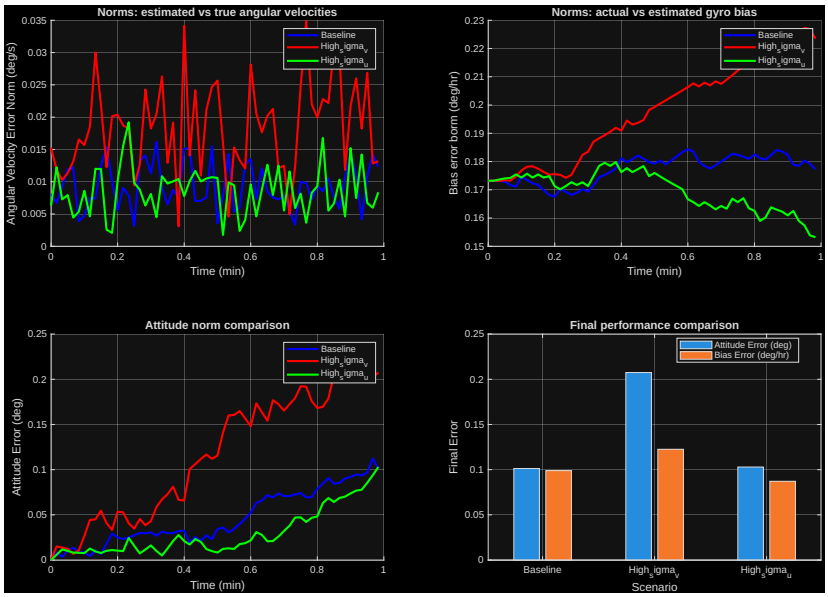
- $\sigma_u = 1 * 10^{-5}$ rad/s/$\sqrt{Hz}$
- $\sigma_v = 2 * 10^{-4}$ rad/s/$\sqrt{Hz}$

0.3 high σ_u

This scenario tests sensitivity to bias instability

- $\sigma_u = 2 * 10^{-5}$ rad/s/$\sqrt{Hz}$
- $\sigma_v = 1 * 10^{-4}$ rad/s/$\sqrt{Hz}$

The following plot ($dt = 1$)



suggests that the filter is more sensitive to increases in measurement noise, than process noise. σ_u was related to the gyro bias drift, and σ_v to the gyro measurement noise. The difference in angular velocities estimates seem to be really stable. If I decrease dt (thus increasing discretization rate) the plot of norms of angular velocities becomes unclear. For the attitude error and gyro bias, the errors seem to diverge more, with the high measurement noise scenario diverging in both cases. So the readings from our sensor, contain a significant amont of random unpredictable error. It does not seem that any increase in process noise has any significant impact on performance, as the norm stays relatively low, at least compared to measurement noise. This is also illustrated by the final comparison plot

1 appendix

1.1 Python code for Exc 3b

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-5, 5, 100)

def plot_convex(a, b, f):
    if b <= a:
        raise ValueError("b must be greater than a")

    y = f(x)
    c = np.random.choice(np.linspace(a, b, 10))

    k = (f(b) - f(a)) / (b - a)
    k_1 = (f(c) - f(a)) / (c - a)
    k_2 = (f(b) - f(c)) / (b - c)

    secant = f(a) + k * (x - a)
    secant_left = f(a) + k_1 * (x - a)
    secant_right = f(c) + k_2 * (x - c)

    plt.figure(figsize=(8, 6))
    plt.plot(x, y, label=r"$f(x)$", color="blue", linewidth=2)
    plt.plot(x, secant, "--", color="orange", alpha=0.8)
    plt.plot(x, secant_left, "--", color="green", alpha=0.8)
    plt.plot(x, secant_right, "--", color="purple", alpha=0.8)

    plt.scatter([a, b, c], [f(a), f(b), f(c)], color="red", s=50)
    plt.annotate(f"a=({a:.1f}, {f(a):.1f})", (a, f(a)))
    plt.annotate(f"b=({b:.1f}, {f(b):.1f})", (b, f(b)))
    plt.annotate(f"c=({c:.1f}, {f(c):.1f})", (c, f(c)))

    x_fill = np.linspace(a, b, 50)
    y_fill = f(x_fill)
    secant_fill = f(a) + k * (x_fill - a)
    plt.fill_between(x_fill, y_fill, secant_fill, alpha=0.2,
            color='red', label='Area showing convexity')

    plt.title("Illustration of Convex Function Properties")
    plt.xlabel("x")
    plt.ylabel("f(x)")
```

```
41        plt.legend()
42        plt.grid(True, linestyle='--', alpha=0.6)
43        plt.axis('equal')
44        plt.show()
45
46    plot_convex(-2, 3, lambda x: x**2)
```

1.2 Matlab code for exercise 4

```
1   nRooms = 5;
2   nControls = 3;
3   nMeasurements = 2;
4
5   q = 1e4;
6   r = 1e2;
7   x_bar = 20 * ones(nRooms, 1);
8   T = 86400;
9   t = linspace(0, T, 1000);
10  c = 1/(1.08e6);
11
12  A = [-30 10 0 0 10;
13       10 -40 10 0 10;
14       0 10 -40 10 10;
15       0 0 10 -30 10;
16       10 10 10 10 -50] * c;
17
18  B = [1 0 0;
19       0 1 0;
20       0 0 0;
21       0 0 1;
22       0 0 0] * c;
23
24  C = [0 1 0 0 0;
25       0 0 0 1 0];
26
27  Q = q * eye(nRooms);
28  R = r * eye(nControls);
29  [K, P, E] = lqr(A, B, Q, R);
30
31  if rank(obsv(A, C)) == nRooms
32      fprintf('System␣is␣observable.\n');
33  else
34      fprintf('System␣is␣NOT␣observable\n');
```

```
35  end
36
37  obs_poles_multiplier = 2;
38  desired_obs_poles = obs_poles_multiplier * max(real(E)) * linspace(1, 1.5,
39  L = place(A', C', desired_obs_poles)';
40
41  B_ambient = [10; 10; 10; 10; 10] * c;
42  D_disturbance = eye(5) * c;
43  x_a = @(t) 20 + 5*sin(2*pi*t/86400);
44  d_func = @(t) max(0, 500 * sin(2 * pi*t/86400) - 300);
45
46  ssign = @(x) (x>0) - (x<0);
47  x0_true = zeros(nRooms, 1);
48  for i = 1:nRooms
49      x0_true(i) = 15 - i * ssign((-1)^i);
50  end
51  z0_true = x0_true - x_bar;
52
53  x0_est = 18 * ones(nRooms, 1);
54  z0_est = x0_est - x_bar;
55
56  z0_aug = [z0_true; z0_est];
57
58  augmentedDynamics = @(t, z_aug) [
59      A * z_aug(1:5) + B * (-K * z_aug(6:10)) + ...
60      B_ambient * (x_a(t) - 20) + ...
61      D_disturbance * d_func(t) * ones(5,1);
62
63      A * z_aug(6:10) + B * (-K * z_aug(6:10)) + ...
64      L * (C * z_aug(1:5) - C * z_aug(6:10))
65  ];
66
67  [t_sim, z_aug] = ode45(@(t,z) augmentedDynamics(t, z), t, z0_aug);
68
69  z_true = z_aug(:, 1:5)';
70  z_est = z_aug(:, 6:10)';
71
72  x_true = z_true + x_bar;
73  x_est = z_est + x_bar;
74
75  u_sim = zeros(length(t_sim), nControls);
76  for i = 1:length(t_sim)
77      u_sim(i,:) = (-K * z_est(:,i))';
78  end
79
```

```
80
81  t_hours = t_sim / 3600;
82
83  figure('Position', [100, 100, 1400, 1000]);
84
85  subplot(3, 2, 1);
86  d_values = arrayfun(d_func, t_sim);
87  x_a_values = arrayfun(x_a, t_sim);
88  plot(t_hours, d_values, 'r-', 'LineWidth', 2); hold on;
89  plot(t_hours, x_a_values, 'b-', 'LineWidth', 2);
90  ylabel('Power (W) / Temp (C)');
91  title('External Influences');
92  legend('Disturbance d(t) [W]', 'Ambient Temp x_a(t) [C]', 'Location', 'bes
93  grid on;
94
95  subplot(3, 2, 2);
96  plot(t_hours, x_true', 'LineWidth', 2); hold on;
97  plot(t_hours, x_est', '--', 'LineWidth', 1);
98  yline(20, '--k', 'Setpoint 20 C', 'LineWidth', 1);
99  ylabel('Temperature (C)');
100 title('True (solid) vs Estimated (dashed) Temperatures');
101 legend('Room 1', 'Room 2', 'Room 3', 'Room 4', 'Room 5', 'Location', 'best
102 grid on;
103
104 subplot(3, 2, 3);
105 plot(t_hours, u_sim, 'LineWidth', 2);
106 ylabel('Heating Power (W)');
107 title('Control Inputs');
108 legend('Room 1 Heater', 'Room 2 Heater', 'Room 4 Heater', 'Location', 'bes
109 grid on;
110
111 subplot(3, 2, 4);
112 est_errors = x_true' - x_est';
113 plot(t_hours, est_errors, 'LineWidth', 1.5);
114 ylabel('Estimation Error (C)');
115 title('Observer Performance: True - Estimated');
116 legend('Room 1', 'Room 2', 'Room 3', 'Room 4', 'Room 5', 'Location', 'best
117 grid on;
118
119 subplot(3, 2, 5);
120 total_power = sum(u_sim, 2);
121 plot(t_hours, total_power, 'k-', 'LineWidth', 2);
122 ylabel('Total Power (W)');
123 xlabel('Time (hours)');
124 title('Total Control Effort');
```

```
125   grid on;
126
127   subplot(3, 2, 6);
128   tracking_errors = x_true' - x_bar';
129   plot(t_hours, tracking_errors, 'LineWidth', 1.5);
130   ylabel('Tracking Error (celsius)');
131   xlabel('Time (hours)');
132   title('True System Tracking Performance');
133   legend('Room 1', 'Room 2', 'Room 3', 'Room 4', 'Room 5', 'Location', 'best
134   grid on;
```

1.3 Matlab code for exc 6

```
1  clear all; close all; clc;
2
3   A = [9.9985 * 10^-1, 9.8510 * 10^-3;
4        -2.9553 * 10^-2, 9.7030 * 10^-1];
5   B = [4.9502 * 10^-5;
6        9.8510 * 10^-3];
7
8   H = [1, 0];
9
10  Q = 1;
11  R = 0.01;
12  N = 1001;
13  dt = 0.01;
14  time = (0:N-1)' * dt;
15
16  x0_true = [1; 1];
17
18  rng(42)
19  w = sqrt(Q) * randn(N,1);
20  v = sqrt(R) * randn(N,1);
21  x_true = zeros(2, N);
22  x_true(:,1) = x0_true;
23
24  for k = 1:N-1
25      x_true(:,k+1) = A * x_true(:,k) + B * w(k);
26  end
27
28  y_meas = H * x_true + v';
29  function [x_hat_plus, P_plus] = runKalmanFilter(A, B, H, Q, R, y_meas, x_h
30      x_hat_minus = zeros(2, N);
```

```
31        x_hat_plus = zeros(2, N);
32        P_minus = zeros(2,2,N);
33        P_plus = zeros(2,2,N);
34
35        x_hat_minus(:,1) = x_hat0_minus;
36        P_minus(:, :, 1) = P0_minus;
37        for k=1:N
38            Kk = P_minus(:, :, k) * H' / (H * P_minus(:, :, k) * H' + R);
39            x_hat_plus(:, k) = x_hat_minus(:, k) + Kk * (y_meas(k) - H * x_hat
40            P_plus(:, :, k) = (eye(2) - Kk * H) * P_minus(:, :, k);
41
42            if k < N
43                x_hat_minus(:, k+1) = A * x_hat_plus(:, k);
44                P_minus(:, :, k+1) = A * P_plus(:, :, k) * A' + B * Q * B';
45            end
46        end
47    end
48
49    [x_hat_plus_1, P_plus_1] = runKalmanFilter(A, B, H, Q, R, y_meas, [1.5; 0.
50    [x_hat_plus_2, P_plus_2] = runKalmanFilter(A, B, H, Q, R, y_meas, [50; -50
51    [x_hat_plus_3, P_plus_3] = runKalmanFilter(A, B, H, Q, R, y_meas, [2; -1],
52    [x_hat_plus_4, P_plus_4] = runKalmanFilter(A, B, H, Q, R, y_meas, [1; 1],
53
54    error_1 = x_true - x_hat_plus_1;
55    error_2 = x_true - x_hat_plus_2;
56    error_3 = x_true - x_hat_plus_3;
57    error_4 = x_true - x_hat_plus_4;
58
59    dist1 = sqrt(sum(error_1.^2, 1));
60    dist2 = sqrt(sum(error_2.^2, 1));
61    dist3 = sqrt(sum(error_3.^2, 1));
62    dist4 = sqrt(sum(error_4.^2, 1));
63
64    figure;
65    plot(time, dist1, 'r-', 'LineWidth', 1.5); hold on;
66    plot(time, dist2, 'g-', 'LineWidth', 1.5);
67    plot(time, dist3, 'b-', 'LineWidth', 1.5);
68    plot(time, dist4, 'm-', 'LineWidth', 1.5);
69    xlabel('Time␣(s)');
70    ylabel('Euclidean␣Distance');
71    title('Distance␣Between␣True␣and␣Estimated␣State␣-␣Interesting␣Scenarios')
72    legend(...
73        'Case␣1:␣Slightly␣Wrong␣+␣Overconfident', ...
74        'Case␣2:␣Very␣Wrong␣+␣Very␣Uncertain', ...
75        'Case␣3:␣Somewhat␣Wrong␣+␣Realistic', ...
```

```
76          'Case␣4:␣Perfect␣+␣Overconfident', ...
77          'Location', 'best');
78      grid on;
79
80      figure('Position', [80, 80, 900, 1200]);
81
82      subplot(4,2,1);
83      sigma_bounds_1_state1 = 3 * squeeze(sqrt(P_plus_1(1,1,:)));
84      plot(time, error_1(1,:), 'b', 'LineWidth', 1.5); hold on;
85      plot(time, sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
86      plot(time, -sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
87      ylabel('Error');
88      title('Case␣1:␣State␣1␣Error␣(x0=[1.5;␣0.5],␣P=[0.0001␣0;␣0␣0.0001])');
89      legend('Error', '3sigma␣Bound', 'Location', 'best');
90      grid on;
91
92      subplot(4,2,2);
93      sigma_bounds_1_state2 = 3 * squeeze(sqrt(P_plus_1(2,2,:)));
94      plot(time, error_1(2,:), 'b', 'LineWidth', 1.5); hold on;
95      plot(time, sigma_bounds_1_state2, 'r--', 'LineWidth', 1);
96      plot(time, -sigma_bounds_1_state2, 'r--', 'LineWidth', 1);
97      ylabel('Error');
98      title('Case␣1:␣State␣2␣Error␣(x0=[1.5;␣0.5],␣P=[0.0001␣0;␣0␣0.0001])');
99      legend('Error', '3sigma␣Bound', 'Location', 'best');
100     grid on;
101
102     subplot(4,2,3);
103     sigma_bounds_2_state1 = 3 * squeeze(sqrt(P_plus_2(1,1,:)));
104     plot(time, error_2(1,:), 'b', 'LineWidth', 1.5); hold on;
105     plot(time, sigma_bounds_2_state1, 'r--', 'LineWidth', 1);
106     plot(time, -sigma_bounds_2_state1, 'r--', 'LineWidth', 1);
107     ylabel('Error');
108     title('Case␣2:␣State␣1␣Error␣(x0=[50;␣-50],␣P=[100␣0;␣0␣100])');
109     legend('Error', '3sigma␣Bound', 'Location', 'best');
110     grid on;
111
112     subplot(4,2,4);
113     sigma_bounds_2_state2 = 3 * squeeze(sqrt(P_plus_2(2,2,:)));
114     plot(time, error_2(2,:), 'b', 'LineWidth', 1.5); hold on;
115     plot(time, sigma_bounds_2_state2, 'r--', 'LineWidth', 1);
116     plot(time, -sigma_bounds_2_state2, 'r--', 'LineWidth', 1);
117     ylabel('Error');
118     title('Case␣2:␣State␣2␣Error␣(x0=[50;␣-50],␣P=[100␣0;␣0␣100])');
119     legend('Error', '3sigma␣Bound', 'Location', 'best');
120     grid on;
```

```
121
122   subplot(4,2,5);
123   sigma_bounds_3_state1 = 3 * squeeze(sqrt(P_plus_3(1,1,:)));
124   plot(time, error_3(1,:), 'b', 'LineWidth', 1.5); hold on;
125   plot(time, sigma_bounds_3_state1, 'r--', 'LineWidth', 1);
126   plot(time, -sigma_bounds_3_state1, 'r--', 'LineWidth', 1);
127   ylabel('Error');
128   title('Case 3: State 1 Error (x0=[2; -1], P=[0.5 0.1; 0.1 0.5])');
129   legend('Error', '3sigma Bound', 'Location', 'best');
130   grid on;
131
132
133   subplot(4,2,6);
134   sigma_bounds_3_state2 = 3 * squeeze(sqrt(P_plus_3(2,2,:)));
135   plot(time, error_3(2,:), 'b', 'LineWidth', 1.5); hold on;
136   plot(time, sigma_bounds_3_state2, 'r--', 'LineWidth', 1);
137   plot(time, -sigma_bounds_3_state2, 'r--', 'LineWidth', 1);
138   ylabel('Error');
139   title('Case 3: State 2 Error (x0=[2; -1], P=[0.5 0.1; 0.1 0.5])');
140   legend('Error', '3sigma Bound', 'Location', 'best');
141   grid on;
142
143   subplot(4,2,7);
144   sigma_bounds_4_state1 = 3 * squeeze(sqrt(P_plus_4(1,1,:)));
145   plot(time, error_4(1,:), 'b', 'LineWidth', 1.5); hold on;
146   plot(time, sigma_bounds_4_state1, 'r--', 'LineWidth', 1);
147   plot(time, -sigma_bounds_4_state1, 'r--', 'LineWidth', 1);
148   ylabel('Error');
149   title('Case 4: State 1 Error (x0=[1; 1], P=[0.001 0; 0 0.001])');
150   legend('Error', '3sigma Bound', 'Location', 'best');
151   grid on;
152
153   subplot(4,2,8);
154   sigma_bounds_4_state2 = 3 * squeeze(sqrt(P_plus_4(2,2,:)));
155   plot(time, error_4(2,:), 'b', 'LineWidth', 1.5); hold on;
156   plot(time, sigma_bounds_4_state2, 'r--', 'LineWidth', 1);
157   plot(time, -sigma_bounds_4_state2, 'r--', 'LineWidth', 1);
158   ylabel('Error');
159   title('Case 4: State 2 Error (x0=[1; 1], P=[0.001 0; 0 0.001])');
160   legend('Error', '3sigma Bound', 'Location', 'best');
161   grid on;
```

1.4 Matlab code for exercise 7

```
clear all; close all; clc;

A = [9.9985 * 10^-1, 9.8510 * 10^-3;
    -2.9553 * 10^-2, 9.7030 * 10^-1];
B = [4.9502 * 10^-5;
    9.8510 * 10^-3];

H = [1, 0; 0 1; 1 1];

Q = 1;
R = diag([0.01 0.01 0.01]);
N = 1001;
dt = 0.01;
time = (0:N-1)' * dt;

x0_true = [1; 1];

rng(42)
w = sqrt(Q) * randn(N,1);
v = sqrtm(R) * randn(3, N);
x_true = zeros(2, N);
x_true(:,1) = x0_true;

for k = 1:N-1
    x_true(:,k+1) = A * x_true(:,k) + B * w(k);
end

y_meas = H * x_true + v;
function [x_hat_plus, P_plus, x_hat_minus, P_minus] = ...
    runKalmanFilter(A, B, H, Q, R, y_meas, x_hat0_minus, P0_minus, N)
    x_hat_minus = zeros(2, N);
    x_hat_plus = zeros(2, N);
    P_minus = zeros(2,2,N);
    P_plus = zeros(2,2,N);

    x_hat_minus(:,1) = x_hat0_minus;
    P_minus(:, :, 1) = P0_minus;
    for k=1:N
        Kk = P_minus(:, :, k) * H' / (H * P_minus(:, :, k) * H' + R);
        x_hat_plus(:, k) = x_hat_minus(:, k) + Kk * (y_meas(:, k) - H * x_
        P_plus(:, :, k) = (eye(2) - Kk * H) * P_minus(:, :, k);

        if k < N
            x_hat_minus(:, k+1) = A * x_hat_plus(:, k);
```

```
45                P_minus(:, :, k+1) = A * P_plus(:, :, k) * A' + B * Q * B';
46            end
47        end
48    end
49
50    [x_hat_plus_1, P_plus_1] = runKalmanFilter(A, B, H, Q, R, y_meas, [1.5; 0.
51    [x_hat_plus_2, P_plus_2] = runKalmanFilter(A, B, H, Q, R, y_meas, [50; -50
52    [x_hat_plus_3, P_plus_3] = runKalmanFilter(A, B, H, Q, R, y_meas, [2; -1],
53    [x_hat_plus_4, P_plus_4] = runKalmanFilter(A, B, H, Q, R, y_meas, [1; 1],
54
55    error_1 = x_true - x_hat_plus_1;
56    error_2 = x_true - x_hat_plus_2;
57    error_3 = x_true - x_hat_plus_3;
58    error_4 = x_true - x_hat_plus_4;
59
60    dist1 = sqrt(sum(error_1.^2, 1));
61    dist2 = sqrt(sum(error_2.^2, 1));
62    dist3 = sqrt(sum(error_3.^2, 1));
63    dist4 = sqrt(sum(error_4.^2, 1));
64
65    figure;
66    plot(time, dist1, 'r-', 'LineWidth', 1.5); hold on;
67    plot(time, dist2, 'g-', 'LineWidth', 1.5);
68    plot(time, dist3, 'b-', 'LineWidth', 1.5);
69    plot(time, dist4, 'm-', 'LineWidth', 1.5);
70    xlabel('Time␣(s)');
71    ylabel('Euclidean␣Distance');
72    title('Distance␣Between␣True␣and␣Estimated␣State␣-␣Interesting␣Scenarios')
73    legend(...
74        'Case␣1:␣Slightly␣Wrong␣+␣Overconfident', ...
75        'Case␣2:␣Very␣Wrong␣+␣Very␣Uncertain', ...
76        'Case␣3:␣Somewhat␣Wrong␣+␣Realistic', ...
77        'Case␣4:␣Perfect␣+␣Overconfident', ...
78        'Location', 'best');
79    grid on;
80
81    figure('Position', [80, 80, 900, 1200]);
82
83    subplot(4,2,1);
84    sigma_bounds_1_state1 = 3 * squeeze(sqrt(P_plus_1(1,1,:)));
85    plot(time, error_1(1,:), 'b', 'LineWidth', 1.5); hold on;
86    plot(time, sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
87    plot(time, -sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
88    ylabel('Error');
89    title('Case␣1:␣State␣1␣Error␣(x0=[1.5;␣0.5],␣P=[0.0001␣0;␣0␣0.0001])');
```

```
90  legend('Error', '3sigma Bound', 'Location', 'best');
91  grid on;
92
93  subplot(4,2,2);
94  sigma_bounds_1_state2 = 3 * squeeze(sqrt(P_plus_1(2,2,:)));
95  plot(time, error_1(2,:), 'b', 'LineWidth', 1.5); hold on;
96  plot(time, sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
97  plot(time, -sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
98  ylabel('Error');
99  title('Case 1: State 2 Error (x0=[1.5; 0.5], P=[0.0001 0; 0 0.0001])');
100 legend('Error', '3sigma Bound', 'Location', 'best');
101 grid on;
102
103 subplot(4,2,3);
104 sigma_bounds_2_state1 = 3 * squeeze(sqrt(P_plus_2(1,1,:)));
105 plot(time, error_2(1,:), 'b', 'LineWidth', 1.5); hold on;
106 plot(time, sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
107 plot(time, -sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
108 ylabel('Error');
109 title('Case 2: State 1 Error (x0=[50; -50], P=[100 0; 0 100])');
110 legend('Error', '3sigma Bound', 'Location', 'best');
111 grid on;
112
113 subplot(4,2,4);
114 sigma_bounds_2_state2 = 3 * squeeze(sqrt(P_plus_2(2,2,:)));
115 plot(time, error_2(2,:), 'b', 'LineWidth', 1.5); hold on;
116 plot(time, sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
117 plot(time, -sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
118 ylabel('Error');
119 title('Case 2: State 2 Error (x0=[50; -50], P=[100 0; 0 100])');
120 legend('Error', '3sigma Bound', 'Location', 'best');
121 grid on;
122
123 subplot(4,2,5);
124 sigma_bounds_3_state1 = 3 * squeeze(sqrt(P_plus_3(1,1,:)));
125 plot(time, error_3(1,:), 'b', 'LineWidth', 1.5); hold on;
126 plot(time, sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
127 plot(time, -sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
128 ylabel('Error');
129 title('Case 3: State 1 Error (x0=[2; -1], P=[0.5 0.1; 0.1 0.5])');
130 legend('Error', '3sigma Bound', 'Location', 'best');
131 grid on;
132
133 subplot(4,2,6);
134 sigma_bounds_3_state2 = 3 * squeeze(sqrt(P_plus_3(2,2,:)));
```

```
135   plot(time, error_3(2,:), 'b', 'LineWidth', 1.5); hold on;
136   plot(time, sigma_bounds_3_state2, 'r--', 'LineWidth', 1);
137   plot(time, -sigma_bounds_3_state2, 'r--', 'LineWidth', 1);
138   ylabel('Error');
139   title('Case 3: State 2 Error (x0=[2; -1], P=[0.5 0.1; 0.1 0.5])');
140   legend('Error', '3sigma Bound', 'Location', 'best');
141   grid on;
142
143   subplot(4,2,7);
144   sigma_bounds_4_state1 = 3 * squeeze(sqrt(P_plus_4(1,1,:)));
145   plot(time, error_4(1,:), 'b', 'LineWidth', 1.5); hold on;
146   plot(time, sigma_bounds_4_state1, 'r--', 'LineWidth', 1);
147   plot(time, -sigma_bounds_4_state1, 'r--', 'LineWidth', 1);
148   ylabel('Error');
149   title('Case 4: State 1 Error (x0=[1; 1], P=[0.001 0; 0 0.001])');
150   legend('Error', '3sigma Bound', 'Location', 'best');
151   grid on;
152
153   subplot(4,2,8);
154   sigma_bounds_4_state2 = 3 * squeeze(sqrt(P_plus_4(2,2,:)));
155   plot(time, error_4(2,:), 'b', 'LineWidth', 1.5); hold on;
156   plot(time, sigma_bounds_4_state2, 'r--', 'LineWidth', 1);
157   plot(time, -sigma_bounds_4_state2, 'r--', 'LineWidth', 1);
158   ylabel('Error');
159   title('Case 4: State 2 Error (x0=[1; 1], P=[0.001 0; 0 0.001])');
160   legend('Error', '3sigma Bound', 'Location', 'best');
161   grid on;
```

1.5 Matlab code for exercise 8

```
1   % Exc 8
2
3   clear; close all; clc;
4   rng(42);
5
6   dt = 1;
7   T = 60;
8   N = T/dt;
9
10  omega_true = [0.001; 0.002; 0.0005];
11
12  r1 = [1;0;0]; r2 = [0;1;0]; r3 = [0;0;1];
13  r = [r1, r2, r3];
```

```
14
15  function S = skew_symmetric(v)
16      S = [0, -v(3), v(2); v(3), 0, -v(1); -v(2), v(1), 0];
17  end
18
19  function q_out = quat_multiply(q1, q2)
20      v1 = q1(1:3); s1 = q1(4);
21      v2 = q2(1:3); s2 = q2(4);
22      q_out = [s1*v2 + s2*v1 + cross(v1, v2); s1*s2 - dot(v1, v2)];
23      q_out = q_out / norm(q_out);
24  end
25
26
27
28  function R = quat_to_dcm(q)
29      q0 = q(4); q1 = q(1); q2 = q(2); q3 = q(3);
30      R = [1-2*(q2^2+q3^2), 2*(q1*q2-q0*q3), 2*(q1*q3+q0*q2);
31          2*(q1*q2+q0*q3), 1-2*(q1^2+q3^2), 2*(q2*q3-q0*q1);
32          2*(q1*q3-q0*q2), 2*(q2*q3+q0*q1), 1-2*(q1^2+q2^2)];
33  end
34
35  function q_conj = quat_conjugate(q)
36      q_conj = [-q(1:3); q(4)];
37  end
38
39  function euler = quat_to_euler(q)
40      q0 = q(4); q1 = q(1); q2 = q(2); q3 = q(3);
41
42      sinr_cosp = 2 * (q0 * q1 + q2 * q3);
43      cosr_cosp = 1 - 2 * (q1 * q1 + q2 * q2);
44      roll = atan2(sinr_cosp, cosr_cosp);
45      sinp = 2 * (q0 * q2 - q3 * q1);
46      if abs(sinp) >= 1
47          pitch = sign(sinp) * pi/2;
48      else
49          pitch = asin(sinp);
50      end
51
52      siny_cosp = 2 * (q0 * q3 + q1 * q2);
53      cosy_cosp = 1 - 2 * (q2 * q2 + q3 * q3);
54      yaw = atan2(siny_cosp, cosy_cosp);
55
56      euler = [roll; pitch; yaw];
57  end
58
```

```
59  function q_out = exact_attitude(q0, omega, t)
60      omega_norm = norm(omega);
61      if omega_norm > 1e-12
62          axis = omega / omega_norm;
63          angle = omega_norm * t;
64          q_rot = [axis * sin(angle/2); cos(angle/2)];
65          q_out = quat_multiply(q0, q_rot);
66      else
67          q_out = q0;
68      end
69      q_out = q_out / norm(q_out);
70  end
71
72  function [q_est, beta_est, P] = attitude_ekf_corrected(r, q0, beta0, P0, g
73      r1 = r(:,1); r2 = r(:,2); r3 = r(:,3);
74      N = size(gyro_meas, 2);
75      q_est = zeros(4, N);
76      beta_est = zeros(3, N);
77      P = zeros(6, 6, N);
78
79      q_est(:,1) = q0;
80      beta_est(:,1) = beta0;
81      P(:,:,1) = P0;
82
83      for k = 1:N-1
84          omega_est = gyro_meas(:,k) - beta_est(:,k);
85          omega_norm = norm(omega_est);
86          if omega_norm > 1e-12
87              psi = sin(0.5 * omega_norm * dt) * omega_est / omega_norm;
88              cos_term = cos(0.5 * omega_norm * dt);
89
90              Omega = [cos_term * eye(3) - skew_symmetric(psi),
psi;
91                       -psi',                                  cos_term];
92
93              q_est(:,k+1) = Omega * q_est(:,k);
94          else
95              q_est(:,k+1) = q_est(:,k);
96          end
97          q_est(:,k+1) = q_est(:,k+1) / norm(q_est(:,k+1));
98          if q_est(4,k+1) < 0
99              q_est(:,k+1) = -q_est(:,k+1);
100         end
101         beta_est(:,k+1) = beta_est(:,k);
102
```

```
103             omega_norm = norm(omega_est);
104             if omega_norm > 1e-12
105                 omega_skew = skew_symmetric(omega_est);
106                 Phi11 = eye(3) - omega_skew * sin(omega_norm * dt)/omega_norm
107                         omega_skew^2 * (1 - cos(omega_norm * dt))/(omega_norm^
108                 Phi12 = omega_skew * (1 - cos(omega_norm * dt))/(omega_norm^2)
109                         eye(3) * dt - omega_skew^2 * (omega_norm * dt - sin(om
110             else
111                 Phi11 = eye(3);
112                 Phi12 = -eye(3) * dt;
113             end
114             Phi21 = zeros(3,3);
115             Phi22 = eye(3);
116
117             Phi = [Phi11, Phi12; Phi21, Phi22];
118
119
120             Q11 = (sigma_v^2 * dt) * eye(3);
121             Q12 = zeros(3,3);
122             Q22 = (sigma_u^2 * dt) * eye(3);
123             Qd = [Q11, Q12; Q12', Q22];
124
125             P_pred = Phi * P(:,:,k) * Phi' + Qd;
126
127             if ~isempty(star_meas) && k < size(star_meas, 2)
128                 q_pred = q_est(:,k+1);
129                 R_pred = quat_to_dcm(q_pred);
130
131                 H = zeros(9, 6);
132                 H(1:3, 1:3) = skew_symmetric(R_pred * r1);
133                 H(4:6, 1:3) = skew_symmetric(R_pred * r2);
134                 H(7:9, 1:3) = skew_symmetric(R_pred * r3);
135
136                 R_meas = (10 * sigma_star)^2 * eye(9);
137
138                 S = H * P_pred * H' + R_meas;
139                 reg_factor = max(1e-6, 1e-4 * dt) * eye(size(S));
140                 S = S + reg_factor;
141                 K = P_pred * H' / (S + 1e-8 * eye(size(S)));
142
143                 b1_pred = R_pred * r1;
144                 b2_pred = R_pred * r2;
145                 b3_pred = R_pred * r3;
146                 y_pred = [b1_pred; b2_pred; b3_pred];
147
```

```
148                 residual = star_meas(:,k+1) - y_pred;
149
150                 if norm(residual) > 10
151                     fprintf('k=%d: Large residual = %.3f, limiting...\n', k, n
152                     residual = residual / norm(residual) * 0.1;
153                 end
154                 delta_x = K * residual;
155
156                 max_att_update = 0.01 * sqrt(dt);
157                 max_bias_update = 1e-8 * dt;
158
159                 delta_x(1:3) = sign(delta_x(1:3)) .* min(abs(delta_x(1:3)), ma
160                 delta_x(4:6) = sign(delta_x(4:6)) .* min(abs(delta_x(4:6)), ma
161
162                 delta_alpha = delta_x(1:3);
163                 delta_beta = delta_x(4:6);
164
165                 P_upd = (eye(6) - K * H) * P_pred;
166                 P_upd = 0.5 * (P_upd + P_upd');
167                 P_upd = P_upd + 1e-12 * eye(6);
168                 P(:,:,k+1) = 0.5 * (P_upd + P_upd');
169
170                 delta_q = [0.5 * delta_alpha; 1];
171                 delta_q = delta_q / norm(delta_q);
172                 q_est(:,k+1) = quat_multiply(delta_q, q_pred);
173                 q_est(:,k+1) = q_est(:,k+1) / norm(q_est(:,k+1));
174                 if q_est(4,k+1) < 0
175                     q_est(:,k+1) = -q_est(:,k+1);
176                 end
177
178
179                 beta_est(:,k+1) = beta_est(:,k+1) + delta_beta;
180
181             else
182                 P(:,:,k+1) = P_pred;
183             end
184         end
185     end
186
187     scenarios = {
188         struct('sigma_u', 1e-5, 'sigma_v', 1e-4, 'name', 'Baseline'),
189         struct('sigma_u', 1e-5, 'sigma_v', 2e-4, 'name', 'High_sigma_v'),
190         struct('sigma_u', 2e-5, 'sigma_v', 1e-4, 'name', 'High_sigma_u')
191     };
192
```

```
193  sigma_star = 0.1 * pi/180;
194
195  results = cell(1,3);
196
197  for scenario_idx = 1:3
198      scenario = scenarios{scenario_idx};
199      fprintf('\n=== Running scenario: %s ===\n', scenario.name);
200
201      q_true_history = zeros(4, N);
202      beta_true_history = zeros(3, N);
203      euler_true_history = zeros(3, N);
204
205      q_true_history(:,1) = [0; 0; 0; 1];
206      true_beta_rad = [0.1; 0.1; 0.1] * pi/180/3600;
207
208      for k = 1:N
209          if k == 1
210              q_true_history(:,k) = [0; 0; 0; 1];
211          else
212              q_true_history(:,k) = exact_attitude(q_true_history(:,k-1), om
213          end
214          beta_true_history(:,k) = true_beta_rad;
215          euler_true_history(:,k) = quat_to_euler(q_true_history(:,k));
216      end
217
218      gyro_meas = zeros(3, N);
219      for k = 1:N
220          eta_v = scenario.sigma_v * randn(3,1);
221          gyro_meas(:,k) = omega_true + beta_true_history(:,k) + eta_v;
222      end
223
224      star_meas = zeros(9, N);
225      for k = 1:N
226          R_true = quat_to_dcm(q_true_history(:,k));
227          noise = sigma_star * randn(9,1);
228
229          b1 = R_true * r1 + noise(1:3);
230          b2 = R_true * r2 + noise(4:6);
231          b3 = R_true * r3 + noise(7:9);
232
233          star_meas(:,k) = [b1/norm(b1); b2/norm(b2); b3/norm(b3)];
234      end
235
236      q0_est = q_true_history(:,1);
237      beta0_est = [0; 0; 0];
```

```
238
239         P0a = (0.1 * pi/180)^2 * dt;
240         P0b = (0.1 * pi/180/3600)^2 * dt;
241         P0 = diag([P0a, P0a, P0a, P0b, P0b, P0b]);
242
243         [q_est, beta_est, P] = attitude_ekf_corrected(r, q0_est, beta0_est, P0
244                                                       scenario.sigma_v, scenario.sigma_u,
245
246         attitude_errors_deg = zeros(1, N);
247         attitude_error_norms = zeros(1, N);
248         euler_errors_deg = zeros(3, N);
249         bias_errors_deg_hr = zeros(3, N);
250         omega_errors_rad = zeros(3, N);
251         omega_error_norms = zeros(1, N);
252
253         for k = 1:N
254             q_err = quat_multiply(quat_conjugate(q_est(:,k)), q_true_history(:
255             angle_err = 2 * acos(abs(q_err(4))) * 180/pi;
256             attitude_errors_deg(k) = min(angle_err, 180-angle_err);
257
258             attitude_error_norms(k) = norm(q_err(1:3)) * 180/pi;
259
260             euler_est = quat_to_euler(q_est(:,k));
261             euler_errors_deg(:,k) = (euler_est - euler_true_history(:,k)) * 18
262
263             bias_error_rad = beta_est(:,k) - beta_true_history(:,k);
264             bias_errors_deg_hr(:,k) = bias_error_rad * (180/pi * 3600);
265             omega_est = gyro_meas(:,k) - beta_est(:,k);
266             omega_errors_rad(:,k) = omega_est - omega_true;
267             omega_error_norms(k) = norm(omega_errors_rad(:,k)) * 180/pi;
268         end
269
270         attitude_variances = zeros(3, N);
271         bias_variances_deg_hr = zeros(3, N);
272
273         for k = 1:N
274             attitude_variances(:,k) = diag(P(1:3,1:3,k)) * (180/pi)^2;
275             bias_variances_deg_hr(:,k) = diag(P(4:6,4:6,k)) * (180/pi * 3600)^
276         end
277
278         results{scenario_idx} = struct();
279         results{scenario_idx}.name = scenario.name;
280         results{scenario_idx}.attitude_errors_deg = attitude_errors_deg;
281         results{scenario_idx}.attitude_error_norms = attitude_error_norms;
282         results{scenario_idx}.euler_errors_deg = euler_errors_deg;
```

```
283         results{scenario_idx}.bias_est_deg_hr = beta_est * (180/pi * 3600);
284         results{scenario_idx}.bias_errors_deg_hr = bias_errors_deg_hr;
285         results{scenario_idx}.attitude_variances = attitude_variances;
286         results{scenario_idx}.bias_variances_deg_hr = bias_variances_deg_hr;
287         results{scenario_idx}.P = P;
288         results{scenario_idx}.omega_error_norms = omega_error_norms;
289         results{scenario_idx}.omega_errors_rad = omega_errors_rad;
290 
291         fprintf('Final attitude error: %.4f deg\n', attitude_errors_deg(end));
292         fprintf('Final attitude error norm: %.4f deg\n', attitude_error_norms(
293         fprintf('Final bias errors: [%.3f, %.3f, %.3f] deg/hr\n', bias_errors_
294     end
295 
296     time_minutes = (0:N-1)*dt/60;
297 
298     colors = {'b', 'r', 'g'};
299     scenario_names = {scenarios{1}.name, scenarios{2}.name, scenarios{3}.name}
300 
301     figure('Position', [100, 100, 1200, 800]);
302 
303     subplot(2,2,1);
304     hold on;
305     for idx = 1:3
306         plot(time_minutes, results{idx}.omega_error_norms, colors{idx}, 'LineW
307     end
308     ylabel('Angular Velocity Error Norm (deg/s)');
309     xlabel('Time (min)');
310     title('Norms: estimated vs true angular velocities');
311     legend('show');
312     grid on;
313 
314     subplot(2,2,2);
315     hold on;
316     for idx = 1:3
317         bias_errors = results{idx}.bias_errors_deg_hr;
318         bias_error_norms = sqrt(sum(bias_errors.^2, 1));
319         plot(time_minutes, bias_error_norms, colors{idx}, 'LineWidth', 2, 'Dis
320     end
321     ylabel('Bias error borm (deg/hr)');
322     xlabel('Time (min)');
323     title('Norms: actual vs estimated gyro bias');
324     legend('show');
325     grid on;
326 
327     subplot(2,2,3);
```

```
328   hold on;
329   for idx = 1:3
330       plot(time_minutes, results{idx}.attitude_errors_deg, colors{idx}, 'Lin
331   end
332   ylabel('Attitude␣Error␣(deg)');
333   xlabel('Time␣(min)');
334   title('Attitude␣norm␣comparison');
335   legend('show');
336   grid on;
337
338   subplot(2,2,4);
339   final_att_errors = [results{1}.attitude_errors_deg(end), results{2}.attitu
340   final_bias_errors = [mean(abs(results{1}.bias_errors_deg_hr(:,end))), ...
341                        mean(abs(results{2}.bias_errors_deg_hr(:,end))), ...
342                        mean(abs(results{3}.bias_errors_deg_hr(:,end)))];
343   bar_data = [final_att_errors; final_bias_errors]';
344   bar(1:3, bar_data);
345   ylabel('Final␣Error');
346   xlabel('Scenario');
347   title('Final␣performance␣comparison');
348   legend('Attitude␣Error␣(deg)', 'Bias␣Error␣(deg/hr)', 'Location', 'best');
349   set(gca, 'XTickLabel', scenario_names);
350   grid on;
```